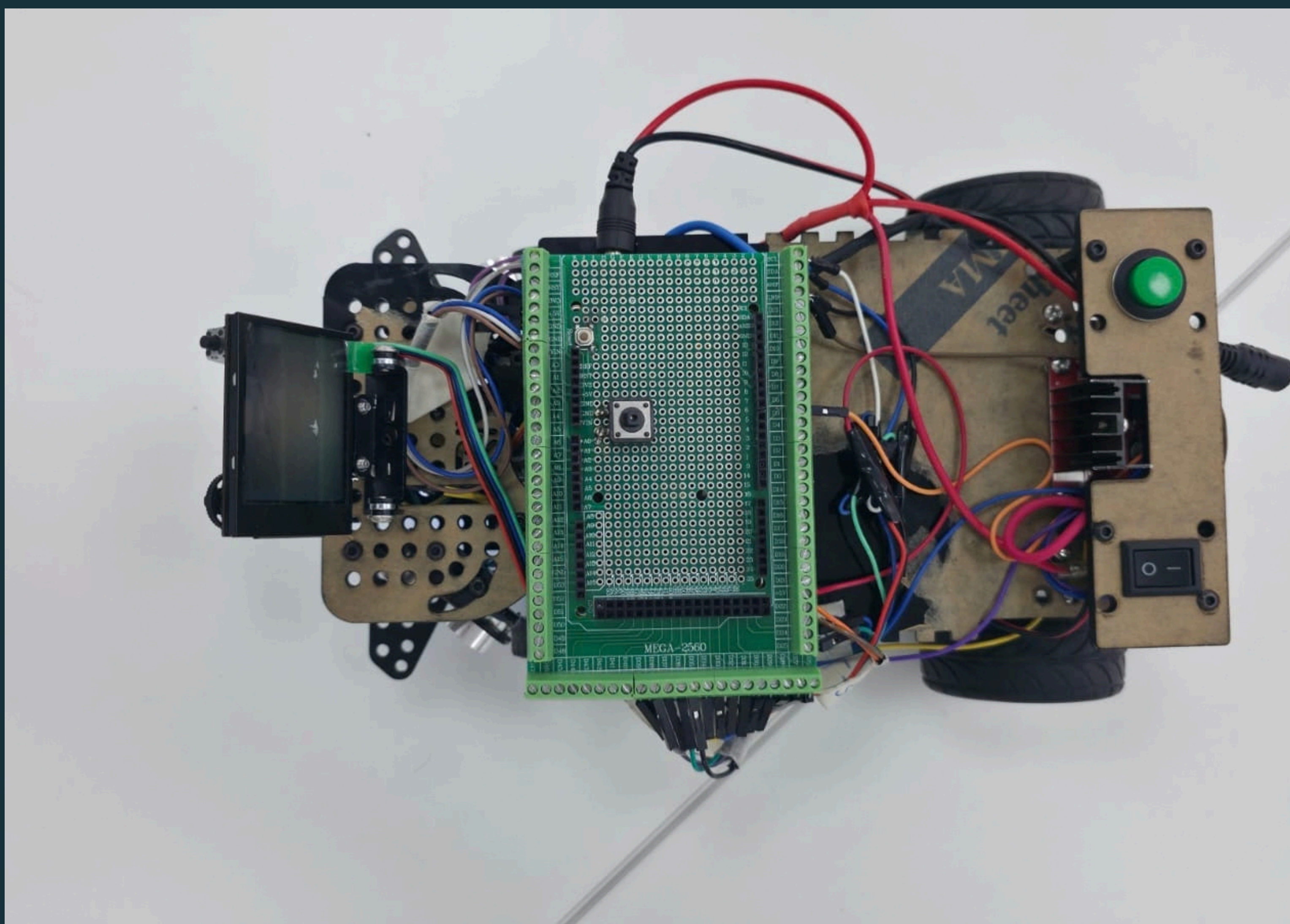


Engineering notebook

WRO 2026

Future Engineers



King Faisal Schools

Mohammed Albassam · Layth Alzahrani

Coach: Eng. Mohamed Salaheldin

Contents

The vehicle, its control program and the evidence behind the build.

Section	Pages
Team and project overview	03-04
Components and mechanical design	05-09
Power and electrical connections	10-12
Sensing and calibration	13-16
Open Challenge software	17-20
Obstacle Challenge software	21-25
HuskyLens future development	26
Testing, repairs and decisions	27-29
Reproduction and project files	30-31
Vehicle photographs	32-33
Sources and technical references	34
Executable source listings	35-42

Document scope

Vehicle design, both challenge programs, the Open Challenge test record and the next stage of HuskyLens development.

Our team

King Faisal Schools



Mohammed Albassam

Mechanical Engineering & Documentation

Mechanical assembly, electronic-board wiring and the engineering record.



Layth Alzahrani

Programming & Electronics

Robot programming, electronic connections and sensor integration.



Eng. Mohamed Salaheldin

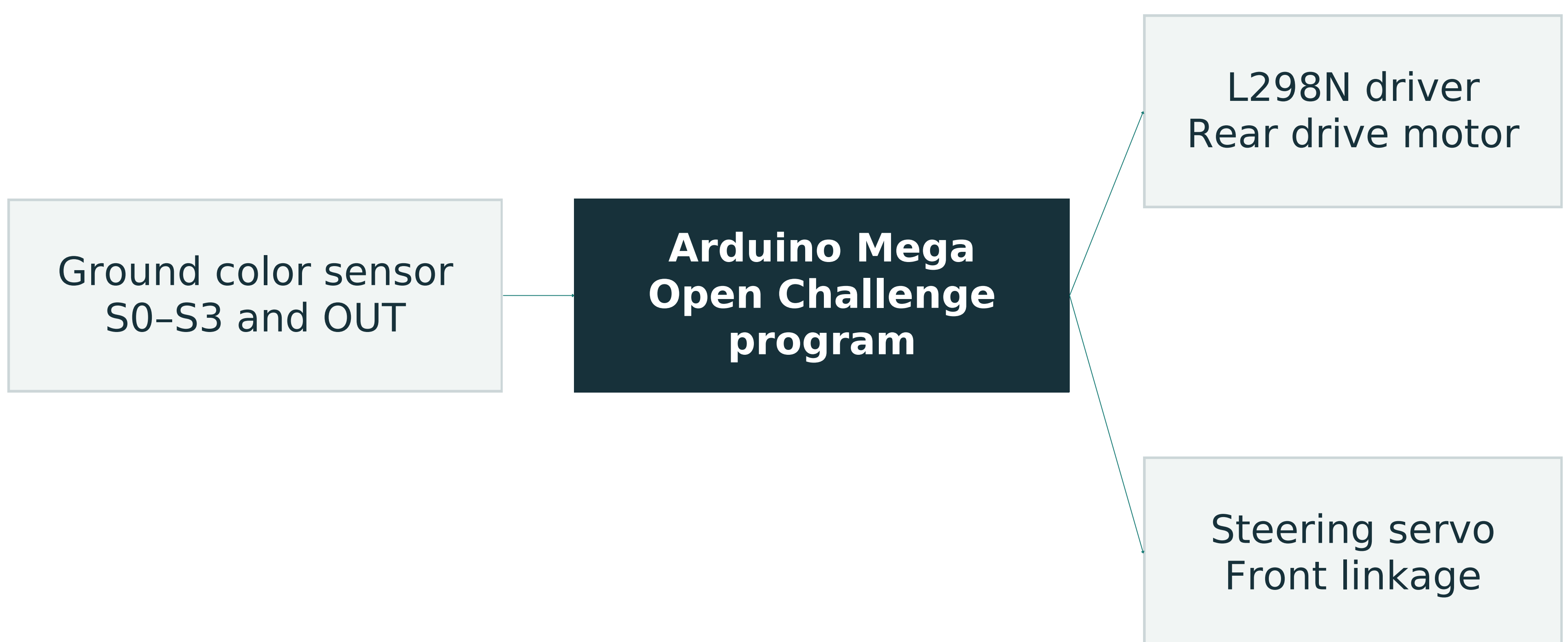
Team Coach & Technical Mentor

Technical guidance, design reviews and support with troubleshooting and preparation.

The vehicle and its control

Rear-wheel propulsion with servo-operated front steering.

We assembled a vehicle around an Arduino Mega, a rear drive motor and a front steering linkage. Open Challenge turns on detected blue floor lines. Obstacle Challenge follows two separately tuned sequences of timed movements.



Control diagram. The start button on D34 releases the program from its waiting state.

Hardware and software boundaries

The vehicle carries five ultrasonic sensors and a HuskyLens 1. Neither submitted sketch reads these devices. Open Challenge uses ground color events; Obstacle Challenge executes a fixed sequence of timed steering and drive commands.

Recorded outcome

The team reports that the Open Challenge program completed three laps and finished correctly. The detailed test record is on page 27.

Component inventory

The main assemblies and their function in the documented vehicle.

Component	Qty	Function
Arduino Mega	1	Main controller
L298N motor-driver module	1	Drive-motor direction and PWM interface
DC drive motor	1	Rear-axle propulsion
Rear differential assembly	1	Allows different rear-wheel speeds in a turn
Steering servo	1	Moves the front steering linkage
Ground color sensor	1	Detects field color beneath the vehicle
Ultrasonic sensors	5	Distance-sensing hardware at five orientations
HuskyLens 1	1	Vision hardware with UART connection
Studica battery	1	12 V nominal, 3000 mAh
Start button	1	D34 input to begin the run
Power switch	1	Power control
Wheels	4	Two rear drive wheels and two front steering wheels

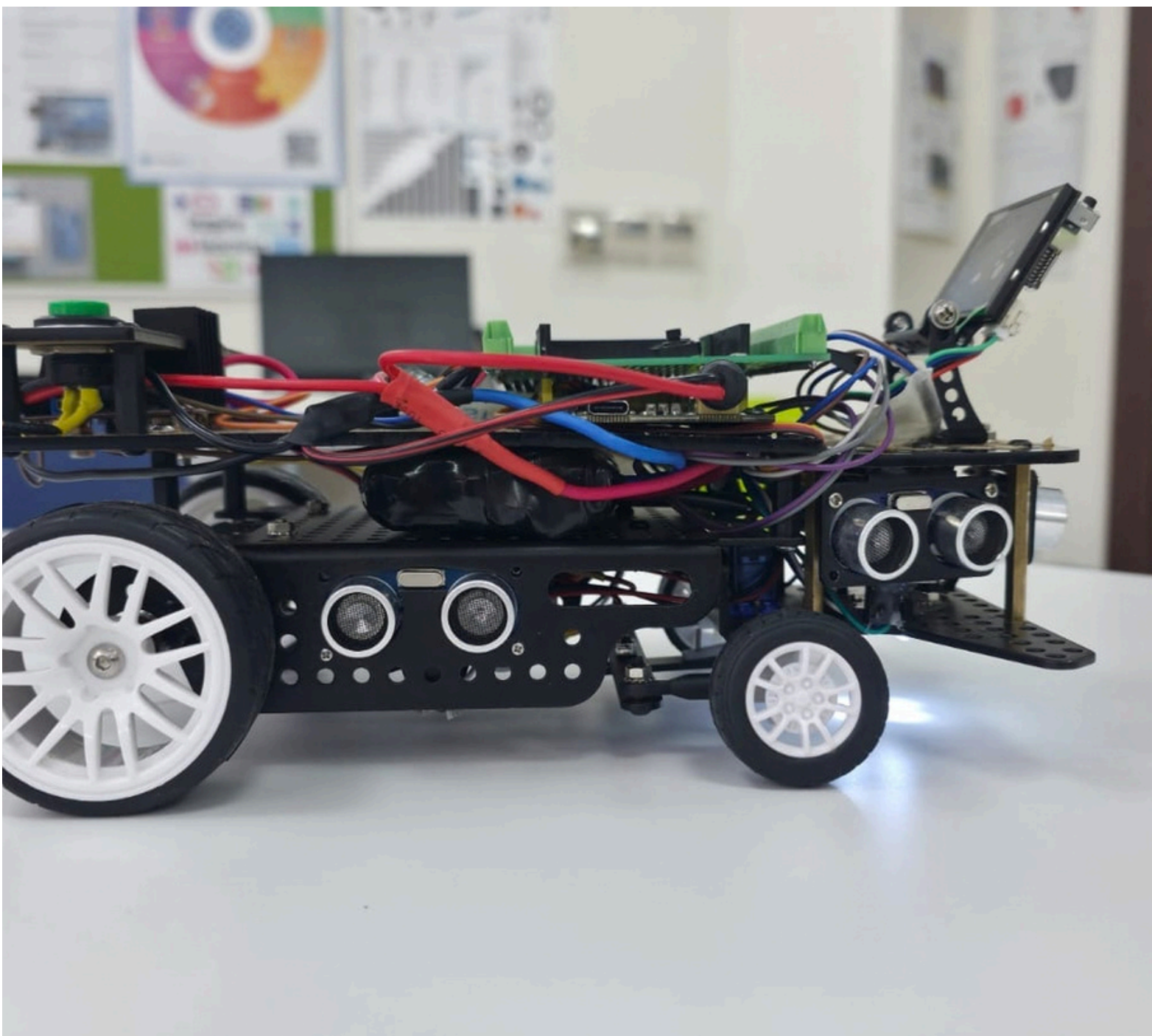
Mechanical platform reference: RN AI manual. Electronics and battery follow the team’s current build.

Dimensions and geometry

Measurements supplied by the team.

Measurement	Centimeters	Millimeters
Overall length	27.5 cm	275 mm
Overall width	19.5 cm	195 mm
Overall height	21.0 cm	210 mm
Wheelbase	17.5 cm	175 mm
Rear-wheel diameter	6.0 cm	60 mm
Front-wheel diameter	3.5 cm	35 mm
Color sensor above the floor	2.75 cm	27.5 mm

Wheelbase is the distance between the front- and rear-wheel centers. Overall width includes the vehicle’s full outer envelope.

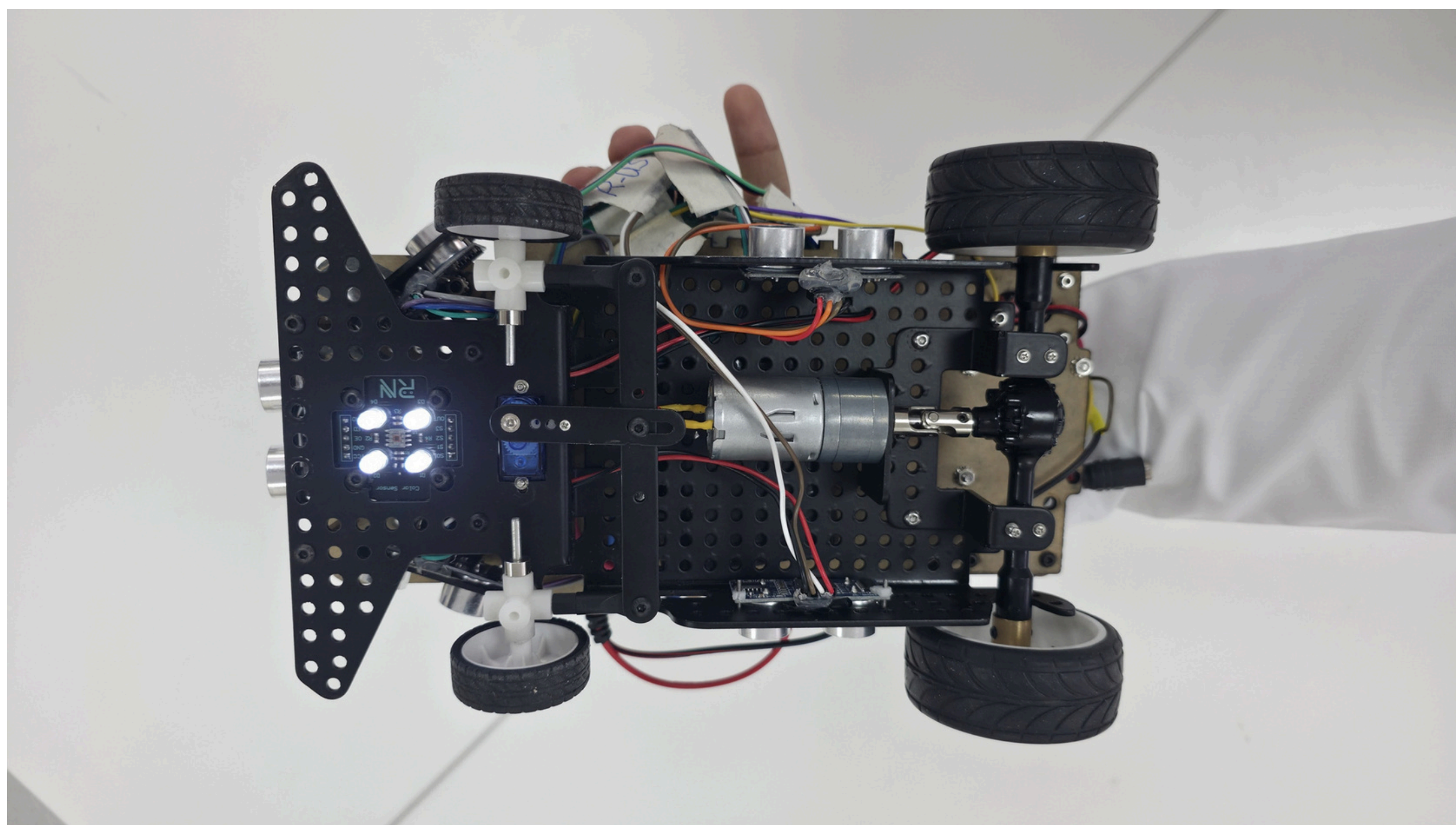


Rear drive and differential

One propulsion motor drives the rear wheels through the differential.

Drive arrangement

The rear wheels supply propulsion. The front wheels steer through the servo linkage. The rear differential allows the outside rear wheel to rotate faster than the inside rear wheel as the vehicle follows a curved path.



Underside view showing the rear drive assembly and running gear.

Wheel size and speed

A 60 mm rear wheel travels approximately 188.5 mm per revolution without slip. Wheel speed can be estimated with $v = \pi \times D \times n / 60$, where D is wheel diameter in meters and n is wheel speed in revolutions per minute.

The kit reference lists a 300 RPM motor. Actual wheel speed depends on the transmission, load and drive command. No measured wheel RPM or vehicle speed is reported here.

Front steering

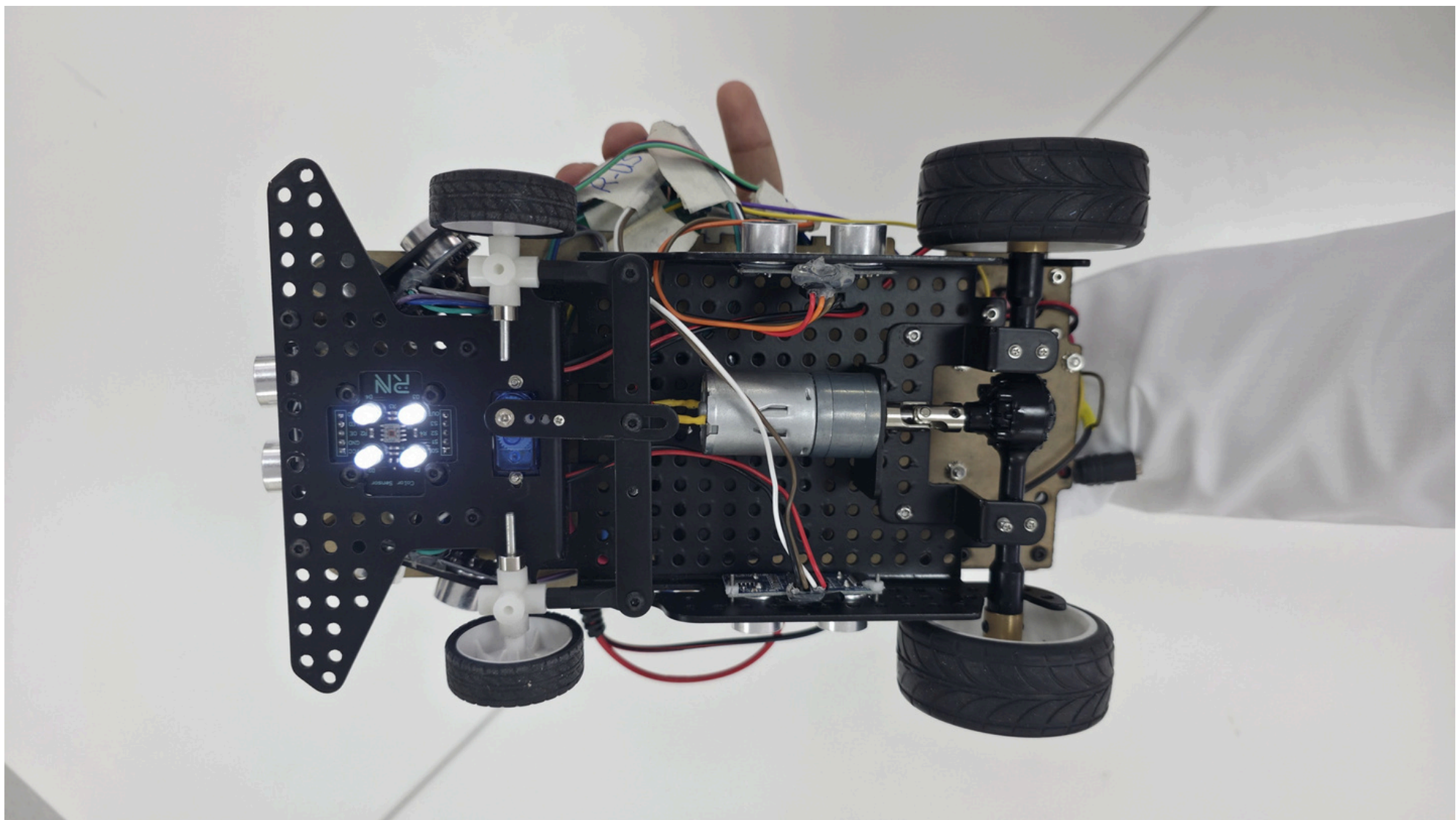
A single servo operates the Ackermann linkage.

How the wheels turn

The linkage gives the inside front wheel a larger steering angle than the outside wheel. This accommodates their different turning radii. The rear differential accommodates the corresponding difference in the paths of the rear wheels.

Program command	Value	Role
STEER_CENTER	82°	Servo command for straight travel
STEER_LEFT	55°	Servo command for a left turn
TURN_TIME_MS	1000 ms	Duration of the commanded turn

These angles are servo commands. They do not directly state the front-wheel angles or the vehicle’s heading change.



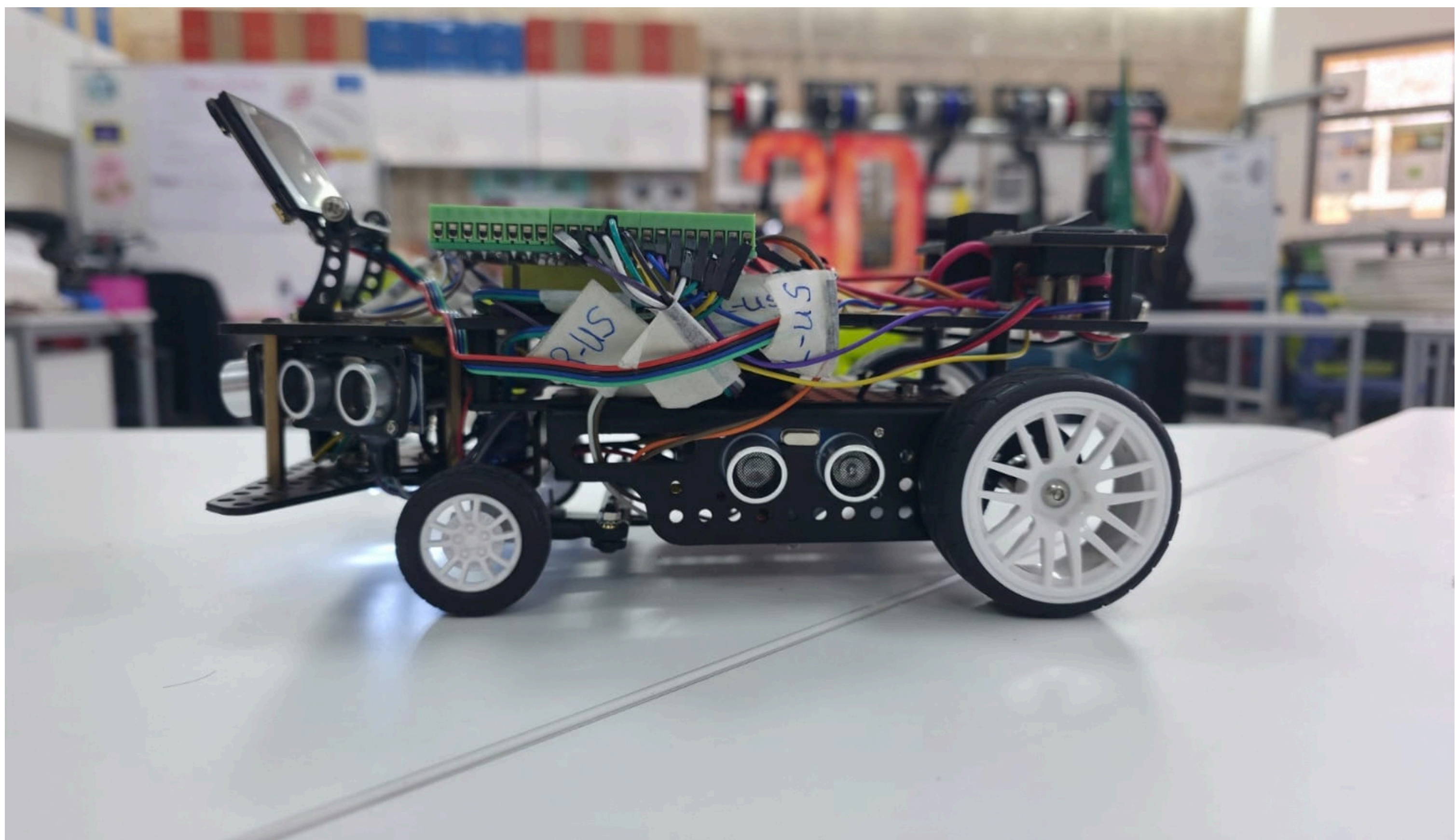
Underside view showing the steering linkage at the front of the vehicle.

Chassis and assembly

Metal running gear with an upper electronics platform.

Structural arrangement

The build combines the RN steering and rear drive assemblies with a metal chassis and an upper electronics platform. The team assembled the mechanical structure and wired the electronic boards. The platform provides mounting space and access to the controller and connections.



Cable clearance

Testing showed that wiring could enter a wheel's moving path. The team replaced the damaged cable and secured the wiring with zip ties. Cable routing must account for wheel rotation and the full movement of the steering linkage.

A useful assembly check is to move the steering through its working range with power off and inspect the clearance around wheels, linkages and cables.

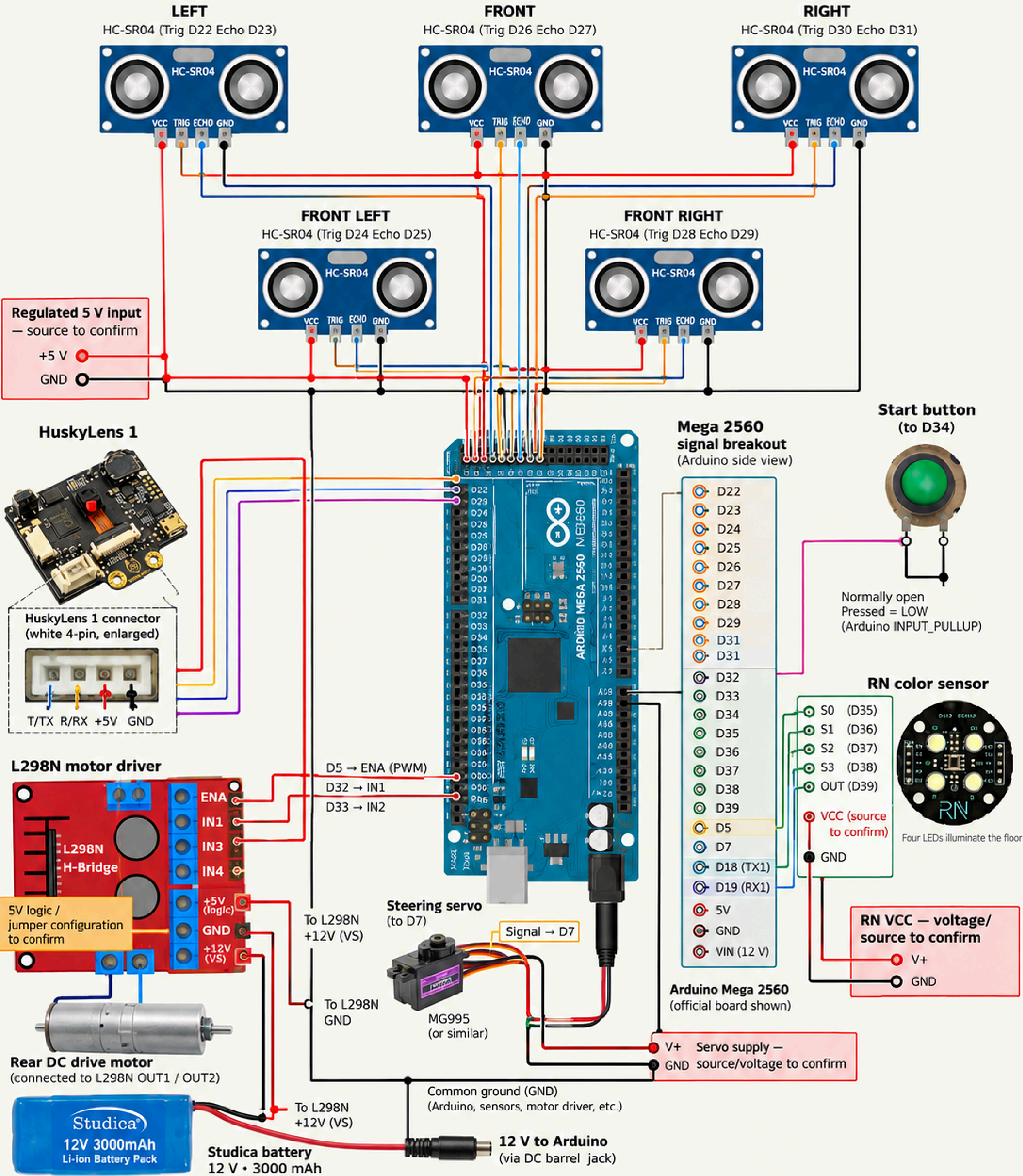
Control and power wiring

KFS Engineers / WRO Future Engineers

ARDUINO MEGA 2560

SIGNALS + COMMON GROUND

DRAFT — supply routing to be confirmed



Wire colour key: TRIG (orange) ECHO (blue) +5 V (red) GND (black) Motor (dark blue) UART (purple) Signal (yellow) Artwork: Fritzing (CC BY-SA 3.0), Yohendry, DFRobot, RN kit manual and team robot photo.

Team-supplied wiring illustration. Confirmed power routes: battery to driver and Mega barrel jack; Arduino to HuskyLens.

Power capacity and load

Battery capacity and regulator loading describe different limits.

Known battery property	Value
Nominal voltage	12 V
Rated capacity	3000 mAh = 3 Ah
Calculated nominal energy	12 V × 3 Ah = 36 Wh

What the capacity tells us

Nominal energy describes the battery’s stored-energy rating. Runtime also depends on average power, usable capacity and losses. A current measurement is needed before quoting an operating time for this vehicle.

What the Arduino regulator must supply

With power entering through the barrel jack, the onboard regulator supplies the Arduino’s 5 V rail. HuskyLens draws power from that rail. The supplied wiring illustration identifies the remaining peripheral supply sources as unconfirmed.

Measurement boundary

Motor current, total current and peak accessory current have not been recorded. This notebook therefore does not assign a numerical runtime, fuse rating or regulator margin.

Electrical connection map

Signal assignments for the Arduino Mega.

Device / signal	Mega connection
Steering servo signal	D7
L298N ENA / drive PWM	D5
L298N IN1 / IN2	D32 / D33
Start button	D34 to GND
Color sensor S0 / S1	D35 / D36
Color sensor S2 / S3	D37 / D38
Color sensor OUT	D39
Left ultrasonic TRIG / ECHO	D22 / D23
Front-left ultrasonic TRIG / ECHO	D24 / D25
Front ultrasonic TRIG / ECHO	D26 / D27
Front-right ultrasonic TRIG / ECHO	D28 / D29
Right ultrasonic TRIG / ECHO	D30 / D31
HuskyLens TX to Mega RX1	D19 / RX1
HuskyLens RX to Mega TX1	D18 / TX1

D34 uses INPUT_PULLUP. Pressing the start button connects it to ground and reads LOW. The Open Challenge sketch uses Serial at 115200 baud for diagnostics. The UART1 and ultrasonic connections belong to the hardware map and are not read by either submitted sketch.

Sensor placement

Each sensor observes a different part of the vehicle’s surroundings.

Sensor	Position / role
Ground color sensor	Below the vehicle, 27.5 mm above the floor
Five ultrasonic sensors	Left, front-left, front, front-right and right
HuskyLens 1	Onboard vision sensor with RX1 / TX1 connection



Role in the documented run

The Open Challenge program reads the ground color sensor. The Obstacle Challenge program uses timed commands without sensor feedback. Time constraints prevented HuskyLens integration into either program. Work on vision integration has now begun.

Ground color measurement

The sketch identifies the frequency-output interface as TCS3200.

The sensor uses S0 and S1 to select output scaling. The program sets S0 HIGH and S1 LOW for 20% scaling. S2 and S3 select one color filter at a time, and D39 receives the output pulses.

Channel	S2	S3
Red	LOW	LOW
Green	HIGH	HIGH
Blue	LOW	HIGH

One sample

After selecting a filter, the program waits 3 ms and measures a LOW pulse with a 30,000 μ s timeout. A successful sample returns 500000 divided by the pulse duration. If no pulse arrives before the timeout, it returns zero.

One color decision

For each filter, the program averages three returned samples. It then classifies the red, green and blue values. These are sequential sensor measurements, not calibrated surface reflectance or standard RGB image values.

Source: readColorChannel(), averageColor() and detectColor() in the uploaded sketch.

Color classification

All ranges are inclusive and come directly from the current sketch.

Condition	BLUE	ORANGE
Red channel R	10000–19000	17500–25500
Green channel G	9000–14500	12000–16500
Blue channel B	11000–20500	13000–19000
Additional test	$B + 3000 \geq R$	$R > B + 3000$
Return value	1	2

Decision order

The program checks BLUE first. A reading must satisfy every channel range and the additional test. If BLUE fails, it checks ORANGE. Readings that satisfy neither condition return OTHER with value zero.

Effect on movement

An armed BLUE result triggers a left turn. ORANGE is reported in the Serial Monitor but does not trigger a turn in this program. ORANGE and OTHER both count as non-blue results for re-arming.

The thresholds describe the team’s calibration. They are not universal values for every field or light level.

Calibration and tuning

The current settings reflect the supplied code and the team’s test account.

Ground-sensor setup

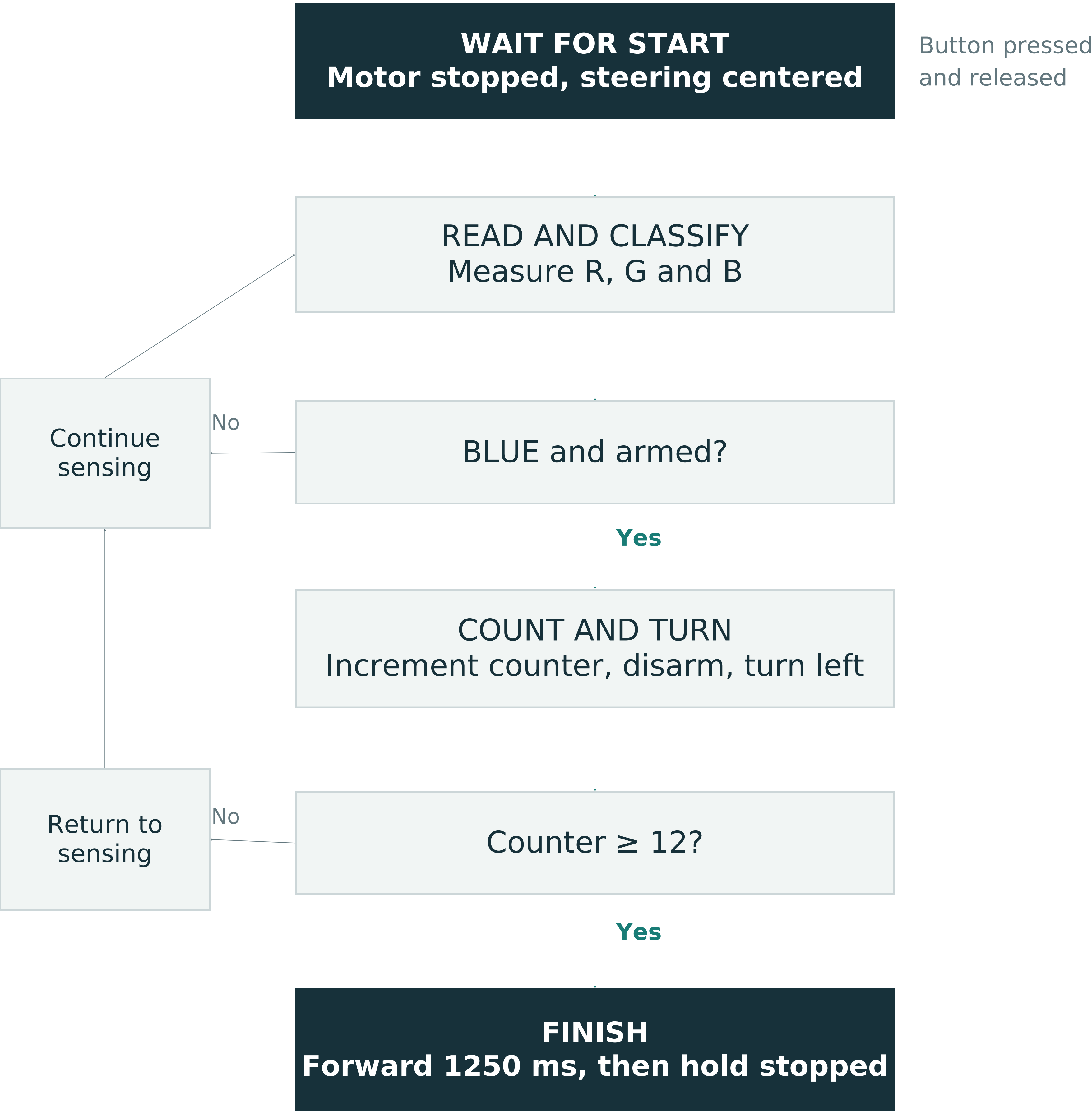
The team measured a mounting height of 2.75 cm. The reported calibration used the sensor’s LEDs, and the team selected thresholds from readings gathered after the first test. Serial output exposes R, G, B and the resulting classification.

Setting	Earlier sketch	Current sketch
Straight steering	80°	82°
Left steering	55°	55°
Straight PWM	170 / 255	140 / 255
Turn PWM	155 / 255	155 / 255
Turn duration	1000 ms	1000 ms
Finish behavior	No active finish routine	12 corners, then 1250 ms forward

The current BLUE ranges are also narrower than the earlier draft. Pages 14–15 record the active acquisition and classification settings. The first two attempts in the reported five-run sequence focused on steering adjustment.

Open Challenge control flow

A behavioral view of the sequential Arduino sketch.



Re-arm rule: three consecutive non-blue results allow the next BLUE trigger. Blue resets that count. During timed turns and the final straight, the program does not sample color.

One trigger for each blue line

The armed flag prevents repeated turns while a blue reading persists.

Trigger handling

A BLUE result always resets nonBlueReadings to zero. If blueArmed is true, the program sets it false, increments cornerCount and calls turnLeft(). Another BLUE result cannot trigger a turn until the program re-arms.

Example reading	Armed state after reading	Turn
BLUE while armed	False	Yes
BLUE again	False	No
OTHER	False, count = 1	No
ORANGE	False, count = 2	No
OTHER	True, count = 3	No
Next BLUE	False	Yes

Illustrative logic sequence, not a recorded sensor trace.

Boundary of the rule

A timeout returns zero and can contribute to a non-blue classification. The three-reading rule reduces repeat triggers, but it does not prove that the vehicle crossed a new physical line or completed a corner.

Counting corners and stopping

The finish routine is active in the uploaded program.

12

accepted blue-line detections
4 corners per lap × 3 laps

The twelfth trigger

The program increments the counter and performs the normal left turn. Only after that turn does it check `cornerCount ≥ 12` and enter `finishRun()`.

Finish stage	Command	Duration
Straighten and advance	82° steering, PWM 140	1250 ms
Stop motor	PWM 0, IN1 LOW, IN2 LOW	Until reset
Hold steering	82°	Until reset

The counter tracks accepted color events. It has no independent position or lap measurement. A missed or false trigger can change the physical stopping location. The final 1.25 seconds is a time command, not a measured travel distance.

Program structure and timing

A single sketch uses the Arduino Servo library.

Function	Responsibility
setup()	Initialize pins and wait for the start-button sequence
driveForward()	Set forward direction and apply the PWM command
stopMotor()	Set PWM to zero and both direction outputs LOW
readColorChannel()	Select a filter and measure one pulse
averageColor()	Average three returned channel measurements
detectColor()	Classify the channels and print diagnostics
turnLeft()	Apply the timed turn, then restore straight commands
finishRun()	Run the final straight and hold the stopped state
loop()	Manage classification, re-arming and corner counting

Blocking intervals

A color decision takes nine samples. The nine filter delays alone total 27 ms. If every pulse measurement reaches its 30 ms timeout, the sampling portion can take about 297 ms, plus execution overhead. Turns block sensing for 1000 ms and the final straight blocks it for 1250 ms.

Obstacle Challenge strategy

Two timed lap sequences on the Arduino Mega.

Implementation under the deadline

We implemented the obstacle route as a sequence of forward movements, gentle steering corrections and stronger left corner commands. Each movement runs for a fixed duration before the next command begins.

Implementation choice	Behavior in the uploaded code
Route execution	runLap() followed immediately by runLap2()
Direction	Forward drive with left corner commands
Sensing	No camera, ultrasonic or ground color measurements
Obstacle handling	Predefined steering movements for the configured route
Completion	Stop after the second sequence and remain stopped

Engineering tradeoff

Timed control allowed us to implement the route within the available time. The resulting path depends on the starting pose, battery condition, traction and the configured obstacle layout. The program does not identify obstacle colors during a run.

Movement commands

Shared drive direction with five movement functions.

Function	Servo	PWM	Purpose
FORWARD(ms)	82°	150	Straight movement
LEFT(ms)	65°	150	Gentle left correction
RIGHT(ms)	92°	150	Gentle right correction
CORNER_LEFT(ms)	55°	220	Stronger left corner command
STOP()	82°	0	Motor stopped, steering centered

Motor and steering outputs

All moving functions set D32 HIGH and D33 LOW for forward drive. D5 provides the PWM command and D7 controls the steering servo. Steering values are servo commands, not measured wheel or vehicle heading angles.

End of a command

Each moving function blocks for delay(ms). CORNER_LEFT() returns the servo to 82° after its delay. STOP() sets PWM to zero and both motor direction pins LOW.

Lap 1: programmed route

Ordered commands in runLap().

F = FORWARD L = LEFT R = RIGHT C = CORNER_LEFT
Numbers are commanded durations in milliseconds.

Route section	Commands in execution order	Total ms
Start side	F1000, C800	1800
Top side	F350, R320, F900, C800	2370
Left side	F300, L250, F350, R500, F400, C800	2600
Bottom side	F400, L220, F800, C500	1920
Final correction	R300, L100, STOP	400

9.09 s

Sum of motion delays in this sequence

The first sequence contains 18 timed movements and four left corner commands. It ends with STOP(), then setup() immediately starts lap two.

Lap 2: programmed route

Ordered commands in runLap2().

F = FORWARD L = LEFT R = RIGHT C = CORNER_LEFT
Numbers are commanded durations in milliseconds.

Route section	Commands in execution order	Total ms
Start side	F600, C800	1400
Top side	F650, R320, F800, C1000	2770
Left side	F300, F600, F400, C800	2100
Bottom side	F400, L220, F800, C500	1920
Final correction	R300, L100, STOP	400

8.59 s

Sum of motion delays in this sequence

The second sequence contains 16 timed movements. LEFT(350), RIGHT(400) and FORWARD(150) remain commented out and do not execute.

Start, run and stop

Execution order and timing from the uploaded sketch.

Stage	Program behavior
Initialize	Attach servo, set motor outputs, enable D34 INPUT_PULLUP and call STOP().
Start input	Wait for the button to read LOW, delay 100 ms, then wait until the button is released.
Start pause	Wait 300 ms after release before beginning the route.
First sequence	Run 18 timed commands totaling 9090 ms, then call STOP().
Second sequence	Immediately run 16 timed commands totaling 8590 ms, then call STOP().
Stopped state	loop() repeats STOP() and a 100 ms delay. Restart requires resetting the controller.

What the timing means

The motion delays total 17.68 s. Including the 300 ms start pause gives 17.98 s after button release, plus execution overhead. These are code-derived durations, not measured completion times.

HuskyLens: next development

Vision integration has begun as future work.

Why we did not use it in this version

Because of the limited time available, we did not integrate the HuskyLens into the challenge programs. We focused on getting the vehicle’s steering, propulsion and route execution working. The hardware remains mounted, and we have already started work on using it in the next version.

What vision will add

Our next objective is to identify red and green obstacles during the run and use their positions in the camera view to guide steering decisions. This will replace the obstacle program’s dependence on a fixed route sequence.

Next implementation step	Intended outcome
Configure and train	Set up reliable red and green obstacle recognition under field lighting.
Read detections	Receive HuskyLens results on the Mega through the documented serial connection.
Connect to motion control	Translate detected obstacle positions into steering decisions and recovery behavior.
Validate on the field	Test different starting positions and obstacle layouts, then record completion, contacts and timing.

Open Challenge testing

Anticlockwise attempts reported by the team.

Attempt	Purpose	Reported outcome
01	Steering adjustment	Used to tune steering angles
02	Steering adjustment	Used to tune steering angles
03	Complete course	Completed three laps
04	Complete course	Completed three laps
05	Complete course	Completed three laps

3 of 3

complete runs after the two steering-adjustment attempts

The team also confirmed that the Open Challenge sketch completed three laps and finished correctly. This is a team-reported outcome. The records do not include measured run times, collision counts or a full matrix of starting positions and wall layouts.

Three successful attempts describe this test sequence. They do not establish performance on every layout.

Wiring failure and repair

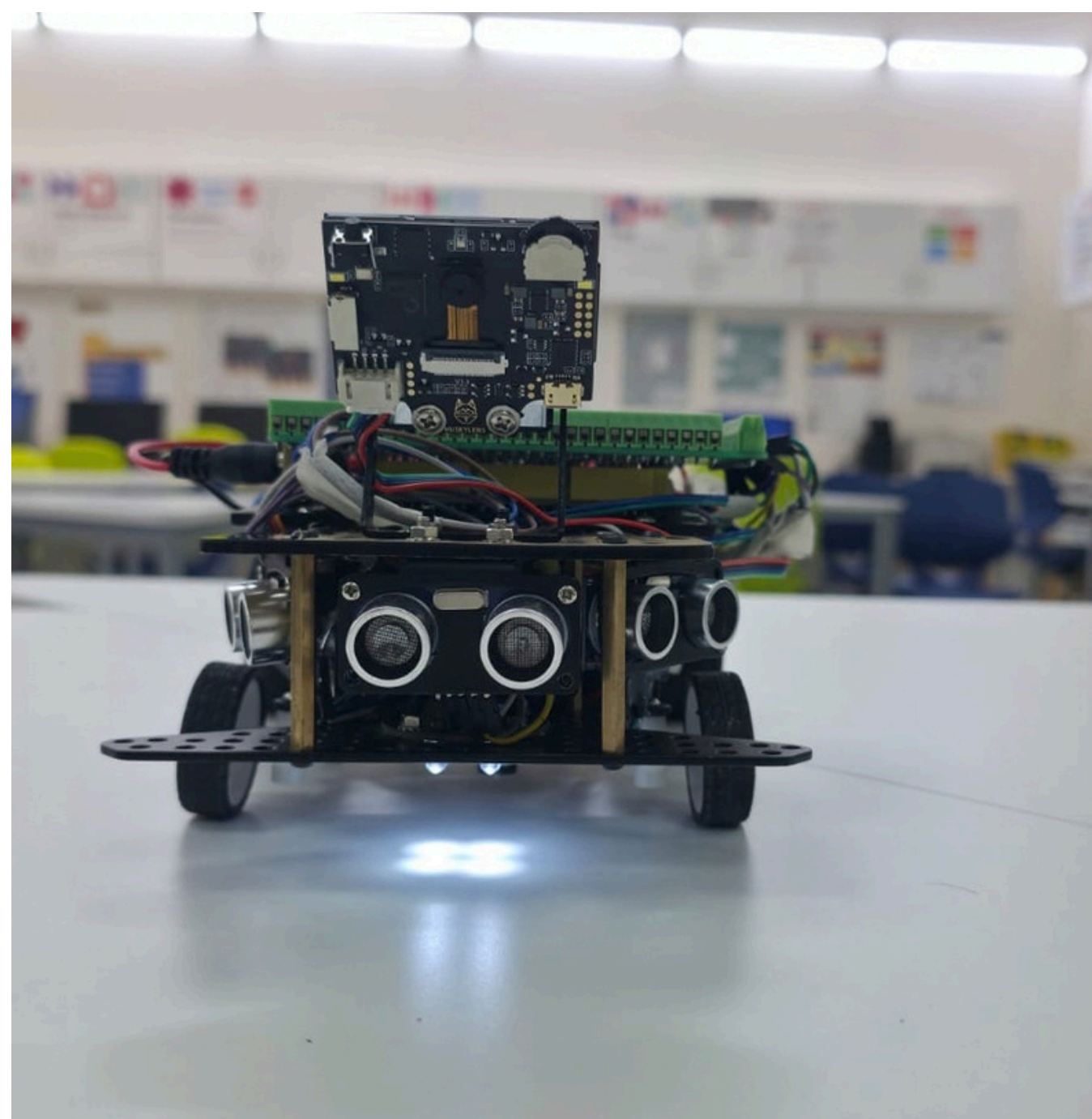
A physical cable problem changed the way the team routed the wiring.

Failure during the first test

An ultrasonic sensor wire entered a wheel's moving path and the wheel cut it. The team replaced the damaged wiring. While doing that work, the Arduino power cord also broke and no spare cord was available.

Repair and securing

The team soldered the power cord and insulated the repaired connection with heat-shrink tubing. They organized the cable routes and used zip ties to keep wires away from moving parts.



The failure involved a sensor that the active Open Challenge program does not read. Its wiring still affected mechanical reliability. This made cable clearance part of the assembly process and maintenance routine.

Engineering decisions

Decisions and constraints supported by the build and test record.

Decision or mechanism	Reasoning and boundary
Separate drive and steering	The rear motor supplies propulsion while the front servo controls direction. Wheel paths in a turn depend on both mechanisms.
Timed left turn	The same angle and duration give a simple repeatable command. The actual path still depends on traction, load and starting alignment.
Three-reading re-arm rule	A persistent BLUE reading cannot repeatedly trigger turns. This is an event filter rather than an independent position check.
12-corner finish routine	The program converts accepted color events into a finish decision. False or missed events affect the count.
Repair and cable restraints	Soldering restored the broken power connection. Heat-shrink and zip ties protect the repaired wiring and its route.

The Obstacle Challenge program adds two timed lap sequences. Both programs leave ultrasonic feedback and HuskyLens unused. Vision integration is the next development priority.

Loading and running the code

Separate sketches: Open Challenge and Obstacle Challenge.

1 Prepare the sketch

Select the required challenge sketch. Place it in a matching Arduino sketch folder and open it in Arduino IDE with the Servo library available.

2 Match the hardware

Use the Mega pin assignments on pages 10–12. Connect the D34 start button to GND. Check polarity and supply connections before powering the vehicle.

3 Upload

Select the Arduino Mega board and its connected port, then compile and upload the sketch. The program starts with the drive stopped and steering centered.

4 Position and start

Place the vehicle for the route represented by the selected sketch. Press and release the start button. Each program pauses for 300 ms after release before beginning forward motion.

5 Observe the finish

Open Challenge stops after 12 accepted blue-line events and its final straight. Obstacle Challenge stops after its two timed sequences. Reset the controller before another run.

For diagnostics, open the Serial Monitor at 115200 baud. The sketch prints channel values and event messages.

Project files and source identity

A clear source record makes the notebook reproducible.

Project repository

github.com/Elzee-Studios/WRO-2026-Future-Engineers

Code used in this edition

This notebook describes both uploaded Arduino sketches. Open Challenge turns on ground color events. Obstacle Challenge runs two timed movement sequences. Their executable listings begin on pages 35 and 40. Both complete original sketches are embedded as PDF attachments.

OPEN CHALLENGE SOURCE SHA-256

3c1d526f4b3143f1d332703a9ae66546
92982e8646753054e87ba630db9ef718

Relationship between the files

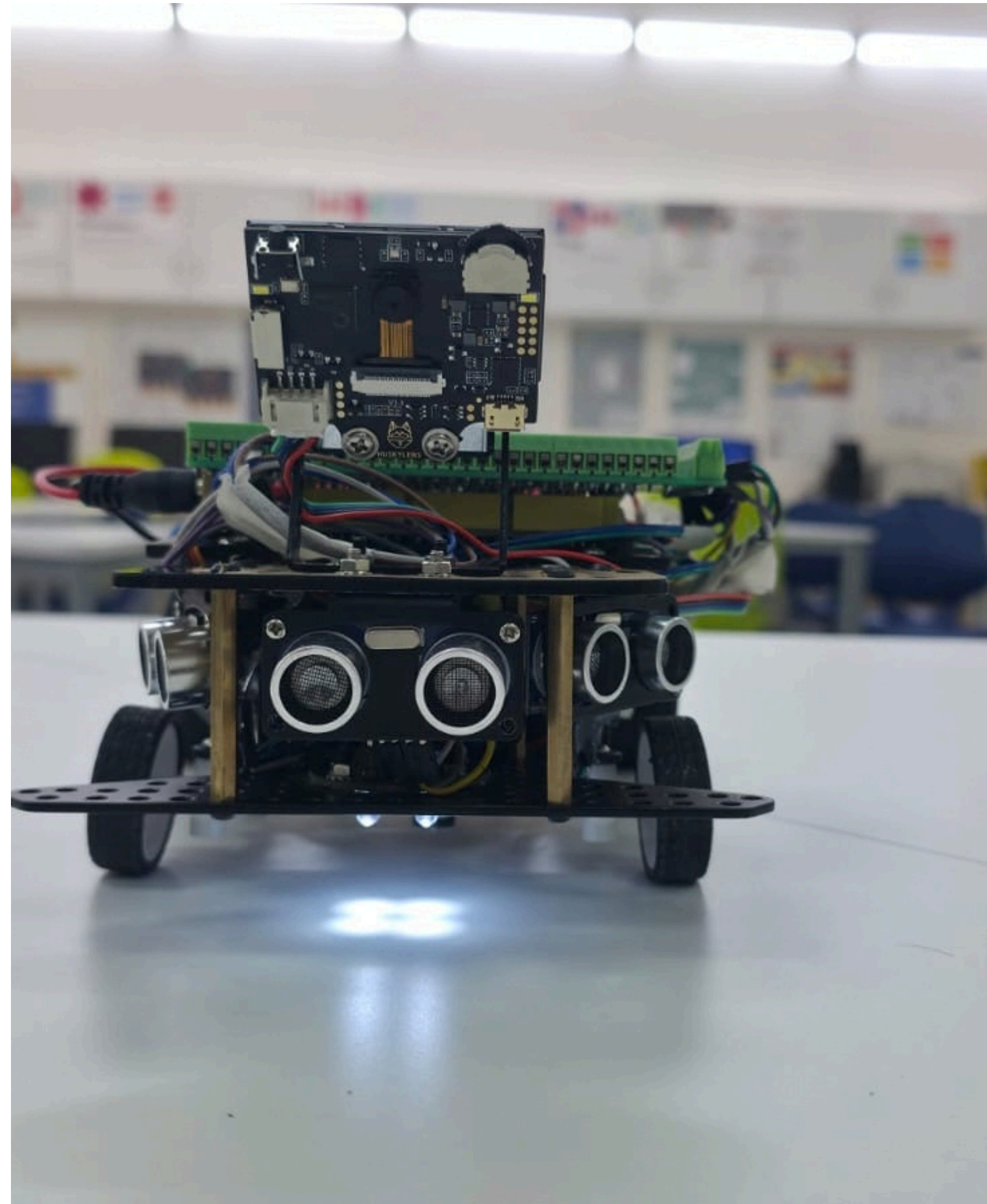
The notebook explains the vehicle and both source files. Electrical connections appear on pages 10–12, Open Challenge control on pages 17–20 and Obstacle Challenge control on pages 21–25. The RN AI manual provides the mechanical assembly reference.

OBSTACLE SOURCE SHA-256

a3912e0577fc7dbaa29c2db6393900dd
dba375d2376abe3915e239833b16d916

Vehicle photo record

Front and rear views of the current vehicle.



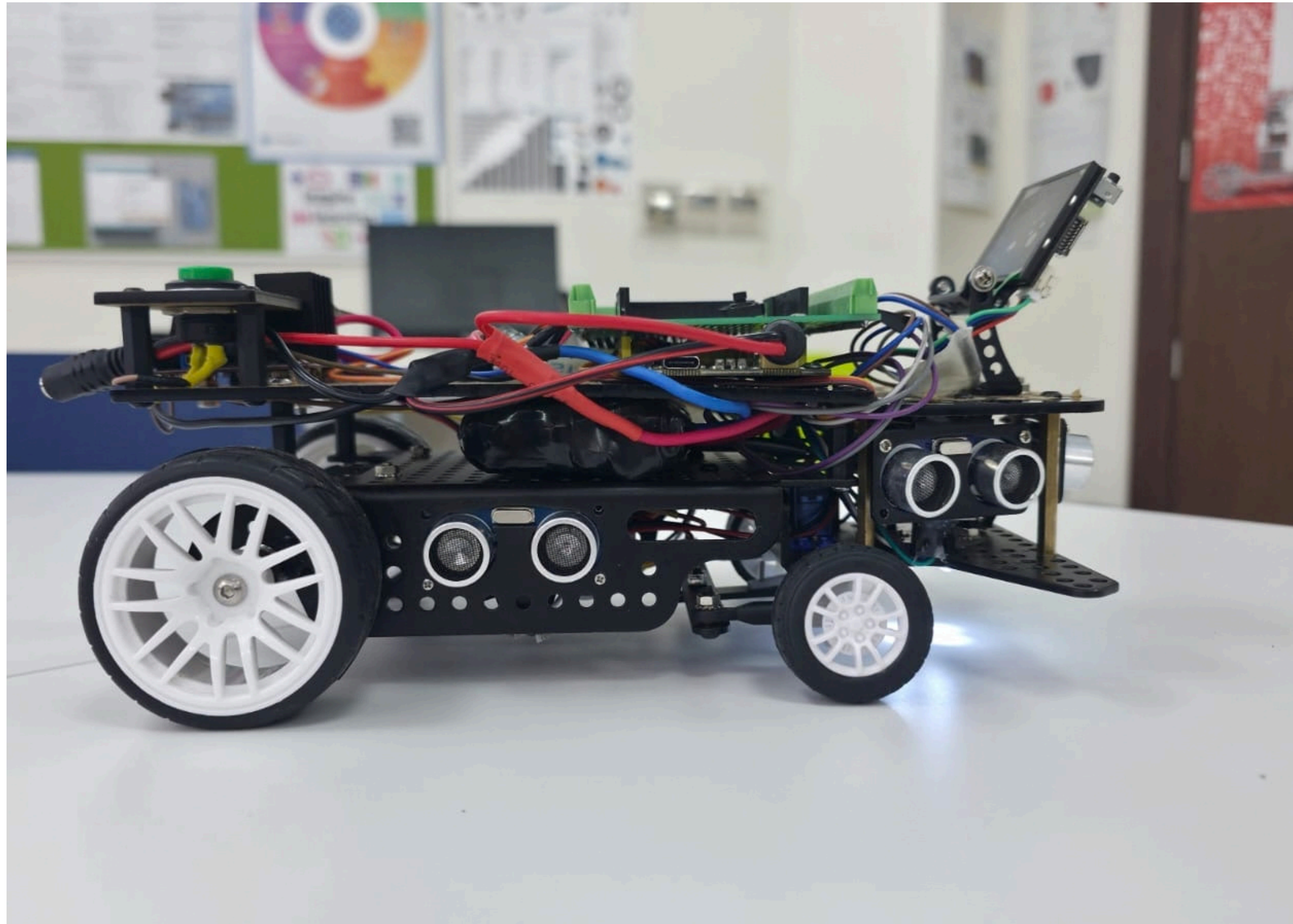
Front view



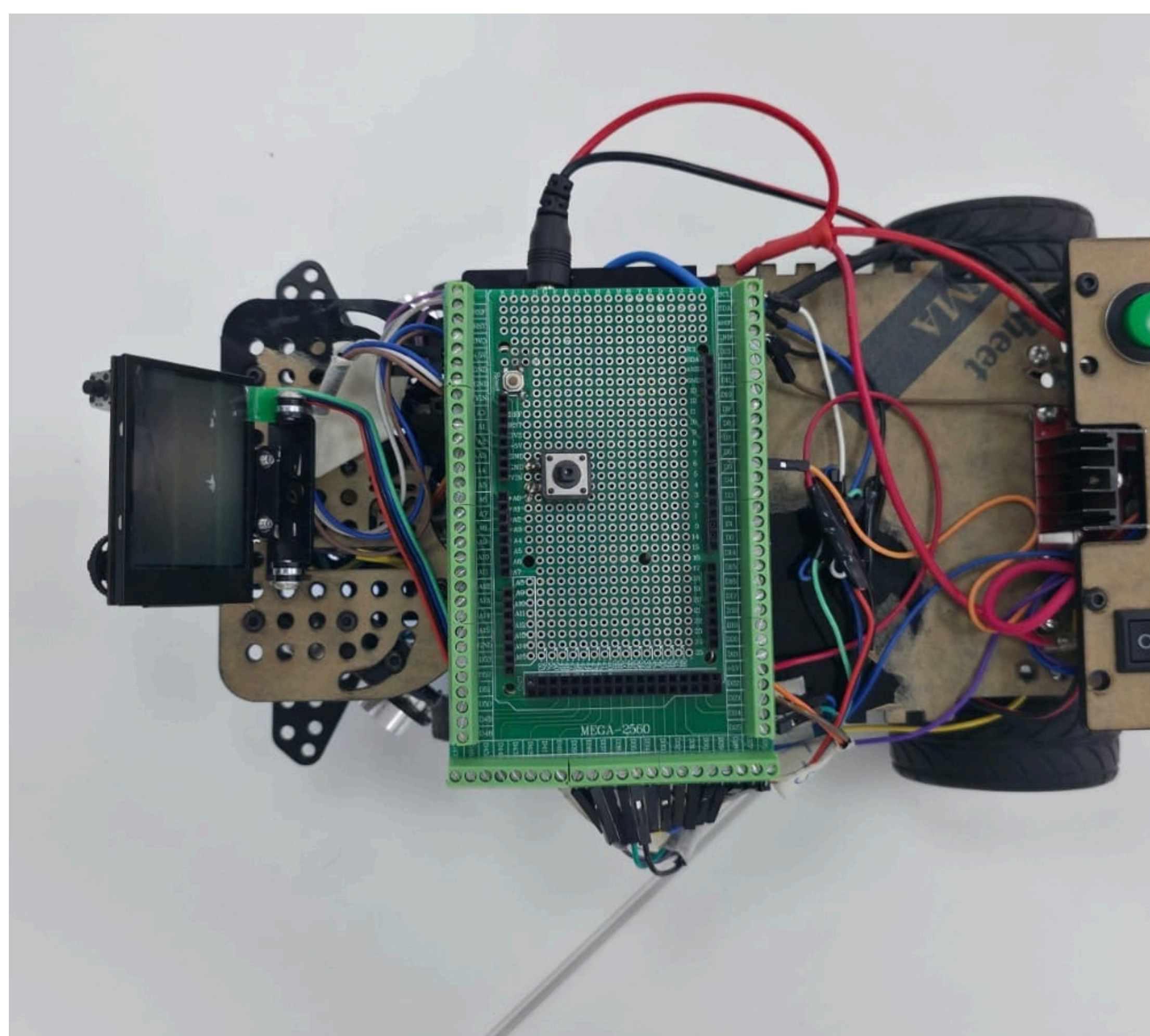
Rear view

Vehicle photo record

Side views of the current vehicle.



Controller and sensor layout



Battery and connection layout

Sources and references

The notebook combines team evidence with technical references.

01 Team build and test record

Team measurements, power-routing confirmations, calibration conditions, wiring repairs, Open Challenge test account, latest robot photographs and the confirmed HuskyLens development plan.

02 Challenge programs

Team-supplied open-challenge-code.ino and obstacle-challenge-code (1).ino. Complete originals are embedded in the PDF. Executable listings begin on pages 35 and 40.

03 Mechanical platform

RN AI User's Manual, supplied as RN 0000000 2000 1.pdf. Mechanical reference: pages 5 and 8–12.

04 Arduino Mega 2560

Board datasheet, pinout and power guidance. docs.arduino.cc/hardware/mega-2560/
docs.arduino.cc/learn/electronics/power-pins/

05 HuskyLens 1

DFRobot SEN0305 documentation.
wiki.dfrobot.com/sen0305/

06 WRO 2026

Future Engineers General & Game Rules and Documentation Rubric. International rules may differ from national arrangements.
wro-association.org/competition/2026-season/

Executable source listing

open-challenge-code.ino

Original source line numbers appear at left. Comments and blank lines are omitted here for compact printing. Executable lines retain their original text and order.

```
1  #include <Servo.h>
8  const int SERVO_PIN = 7;
11 const int MOTOR_PWM = 5;
12 const int MOTOR_IN1 = 32;
13 const int MOTOR_IN2 = 33;
17 const int START_BUTTON = 34;
20 const int S0 = 35;
21 const int S1 = 36;
22 const int S2 = 37;
23 const int S3 = 38;
24 const int COLOR_OUT = 39;
33 int cornerCount = 0;
37 const unsigned long FINISH_DRIVE_MS = 1250;
40 const int STEER_CENTER = 82;
41 const int STEER_LEFT = 55;
44 const int NORMAL_SPEED = 140;
45 const int TURN_SPEED = 155;
49 const unsigned long TURN_TIME_MS = 1000;
56 Servo steering;
60 bool blueArmed = true;
64 int nonBlueReadings = 0;
72 void driveForward(int speedValue) {
74     digitalWrite(MOTOR_IN1, HIGH);
75     digitalWrite(MOTOR_IN2, LOW);
77     analogWrite(MOTOR_PWM, speedValue);
78 }
82 void stopMotor() {
84     analogWrite(MOTOR_PWM, 0);
86     digitalWrite(MOTOR_IN1, LOW);
87     digitalWrite(MOTOR_IN2, LOW);
88 }
100 unsigned long readColorChannel(bool s2, bool s3) {
102     digitalWrite(S2, s2);
103     digitalWrite(S3, s3);
106     delay(3);
108     unsigned long pulse =
109         pulseIn(COLOR_OUT, LOW, 30000);
112     if (pulse == 0) {
113         return 0;
114     }
```

Executable source listing

open-challenge-code.ino

Original source line numbers appear at left. Comments and blank lines are omitted here for compact printing. Executable lines retain their original text and order.

```
116     return 500000UL / pulse;
117 }
122 unsigned long averageColor(bool s2, bool s3) {
124     unsigned long total = 0;
126     for (int i = 0; i < 3; i++) {
128         total += readColorChannel(s2, s3);
129     }
131     return total / 3;
132 }
144 int detectColor() {
147     unsigned long R =
148         averageColor(LOW, LOW);
151     unsigned long G =
152         averageColor(HIGH, HIGH);
155     unsigned long B =
156         averageColor(LOW, HIGH);
160     Serial.print("R: ");
161     Serial.print(R);
163     Serial.print("  G: ");
164     Serial.print(G);
166     Serial.print("  B: ");
167     Serial.print(B);
169     Serial.print("  ---> ");
184     if (
185         R >= 10000 &&
186         R <= 19000 &&
188         G >= 9000 &&
189         G <= 14500 &&
191         B >= 11000 &&
192         B <= 20500 &&
194         B + 3000 >= R
195     )
196     {
197         Serial.println("BLUE");
199         return 1;
200     }
210     if (
211         R >= 17500 &&
212         R <= 25500 &&
214         G >= 12000 &&
```

Executable source listing

open-challenge-code.ino

Original source line numbers appear at left. Comments and blank lines are omitted here for compact printing. Executable lines retain their original text and order.

```
215     G <= 16500 &&
217     B >= 13000 &&
218     B <= 19000 &&
220     R > B + 3000
221 )
222 {
223     Serial.println("ORANGE");
225     return 2;
226 }
230     Serial.println("OTHER");
232     return 0;
233 }
240 void turnLeft() {
242     Serial.println();
243     Serial.println(">>> BLUE DETECTED");
244     Serial.println(">>> STARTING LEFT TURN");
248     steering.write(STEER_LEFT);
252     driveForward(TURN_SPEED);
256     delay(TURN_TIME_MS);
260     steering.write(STEER_CENTER);
264     driveForward(NORMAL_SPEED);
267     Serial.println(">>> TURN COMPLETE");
268     Serial.println();
269 }
276 void finishRun() {
278     Serial.println();
279     Serial.println("=====");
280     Serial.println(">>> 3 LAPS COMPLETE");
281     Serial.println(">>> MOVING INTO FINISH AREA");
282     Serial.println("=====");
286     steering.write(STEER_CENTER);
290     driveForward(NORMAL_SPEED);
294     delay(FINISH_DRIVE_MS);
298     stopMotor();
302     steering.write(STEER_CENTER);
305     Serial.println(">>> STOPPED");
309     while (true) {
311         steering.write(STEER_CENTER);
313         stopMotor();
315         delay(100);
```

Executable source listing

open-challenge-code.ino

Original source line numbers appear at left. Comments and blank lines are omitted here for compact printing. Executable lines retain their original text and order.

```
316     }
317 }
324 void setup() {
326     Serial.begin(115200);
333     steering.attach(SERVO_PIN);
336     steering.write(STEER_CENTER);
343     pinMode(MOTOR_PWM, OUTPUT);
344     pinMode(MOTOR_IN1, OUTPUT);
345     pinMode(MOTOR_IN2, OUTPUT);
348     stopMotor();
355     pinMode(S0, OUTPUT);
356     pinMode(S1, OUTPUT);
358     pinMode(S2, OUTPUT);
359     pinMode(S3, OUTPUT);
361     pinMode(COLOR_OUT, INPUT);
365     digitalWrite(S0, HIGH);
366     digitalWrite(S1, LOW);
373     pinMode(START_BUTTON, INPUT_PULLUP);
377     cornerCount = 0;
380     Serial.println();
381     Serial.println("READY - PRESS BUTTON");
385     steering.write(STEER_CENTER);
389     while (digitalRead(START_BUTTON) == HIGH) {
391         steering.write(STEER_CENTER);
392     }
396     steering.write(STEER_CENTER);
398     delay(100);
402     while (digitalRead(START_BUTTON) == LOW) {
404         steering.write(STEER_CENTER);
405     }
409     delay(300);
412     Serial.println("START");
413     Serial.println("CORNER 0 / 12");
417     steering.write(STEER_CENTER);
419     driveForward(NORMAL_SPEED);
420 }
427 void loop() {
430     int color = detectColor();
437     if (color == 1) {
440         nonBlueReadings = 0;
```

Executable source listing

open-challenge-code.ino

Original source line numbers appear at left. Comments and blank lines are omitted here for compact printing. Executable lines retain their original text and order.

```
444     if (blueArmed) {
446         blueArmed = false;
450         cornerCount++;
453         Serial.print("CORNER: ");
454         Serial.print(cornerCount);
455         Serial.println(" / 12");
459         turnLeft();
463         if (cornerCount >= 12) {
465             finishRun();
466         }
467     }
468 }
475 else {
477     nonBlueReadings++;
485     if (nonBlueReadings >= 3) {
487         blueArmed = true;
488     }
493     steering.write(STEER_CENTER);
495     driveForward(NORMAL_SPEED);
496 }
497 }
```

Obstacle Challenge source

obstacle-challenge-code (1).ino

Original line numbers appear at left. Comments and blank lines are omitted for compact printing. Executable lines retain their original text and order.

```
1      #include <Servo.h>
7      Servo steering;
10     const int SERVO_PIN = 7;
13     const int MOTOR_PWM = 5;
14     const int MOTOR_IN1 = 32;
15     const int MOTOR_IN2 = 33;
19     const int START_BUTTON = 34;
37     void FORWARD(unsigned long ms) {
39         steering.write(82);
41         digitalWrite(MOTOR_IN1, HIGH);
42         digitalWrite(MOTOR_IN2, LOW);
44         analogWrite(MOTOR_PWM, 150);
46         delay(ms);
47     }
62     void LEFT(unsigned long ms) {
64         steering.write(65);
66         digitalWrite(MOTOR_IN1, HIGH);
67         digitalWrite(MOTOR_IN2, LOW);
69         analogWrite(MOTOR_PWM, 150);
71         delay(ms);
72     }
86     void RIGHT(unsigned long ms) {
88         steering.write(92);
90         digitalWrite(MOTOR_IN1, HIGH);
91         digitalWrite(MOTOR_IN2, LOW);
93         analogWrite(MOTOR_PWM, 150);
95         delay(ms);
96     }
112    void CORNER_LEFT(unsigned long ms) {
114        steering.write(55);
116        digitalWrite(MOTOR_IN1, HIGH);
117        digitalWrite(MOTOR_IN2, LOW);
119        analogWrite(MOTOR_PWM, 220);
121        delay(ms);
124        steering.write(82);
125    }
135    void STOP() {
137        analogWrite(MOTOR_PWM, 0);
139        digitalWrite(MOTOR_IN1, LOW);
140        digitalWrite(MOTOR_IN2, LOW);
```

Obstacle Challenge source

obstacle-challenge-code (1).ino

Original line numbers appear at left. Comments and blank lines are omitted for compact printing. Executable lines retain their original text and order.

```
142     steering.write(82);
143 }
155 void runLap() {
162     FORWARD(1000);
165     CORNER_LEFT(800);
178     FORWARD(350);
181     RIGHT(320);
184     FORWARD(900);
187     CORNER_LEFT(800);
198     FORWARD(300);
208     LEFT(250);
211     FORWARD(350);
214     RIGHT(500);
217     FORWARD(400);
220     CORNER_LEFT(800);
230     FORWARD(400);
237     LEFT(220);
240     FORWARD(800);
244     CORNER_LEFT(500);
255     RIGHT(300);
258     LEFT(100);
262     STOP();
263 }
274 void runLap2() {
280     FORWARD(600);
282     CORNER_LEFT(800);
293     FORWARD(650);
296     RIGHT(320);
299     FORWARD(800);
302     CORNER_LEFT(1000);
309     FORWARD(300);
322     FORWARD(600);
329     FORWARD(400);
333     CORNER_LEFT(800);
340     FORWARD(400);
347     LEFT(220);
350     FORWARD(800);
354     CORNER_LEFT(500);
366     RIGHT(300);
368     LEFT(100);
```

Obstacle Challenge source

obstacle-challenge-code (1).ino

Original line numbers appear at left. Comments and blank lines are omitted for compact printing. Executable lines retain their original text and order.

```
371     STOP();
372 }
388 void setup() {
391     steering.attach(SERVO_PIN);
394     steering.write(82);
398     pinMode(MOTOR_PWM, OUTPUT);
399     pinMode(MOTOR_IN1, OUTPUT);
400     pinMode(MOTOR_IN2, OUTPUT);
404     pinMode(START_BUTTON, INPUT_PULLUP);
408     STOP();
419     while (digitalRead(START_BUTTON) == HIGH) {
421         steering.write(82);
422     }
425     delay(100);
432     while (digitalRead(START_BUTTON) == LOW) {
434         steering.write(82);
435     }
439     delay(300);
446     runLap();
448     runLap2();
449 }
460 void loop() {
462     STOP();
464     delay(100);
465 }
```